

Object Oriented Programming



03: Object Oriented Programming

1. Procedural and Object-Oriented Programming
2. Introduction to Classes
3. Reference and Pointer Objects
4. Constructors / Destructors
5. Copy Constructors (Shallow / Deep Copy)
6. Static Class Variables and Functions
7. This Pointer
8. Member Initializer List
9. Friend Functions and Friend Classes



Dr. Imran Ihsan
Ph.D. in Knowledge Engineering
Associate Professor





03-01

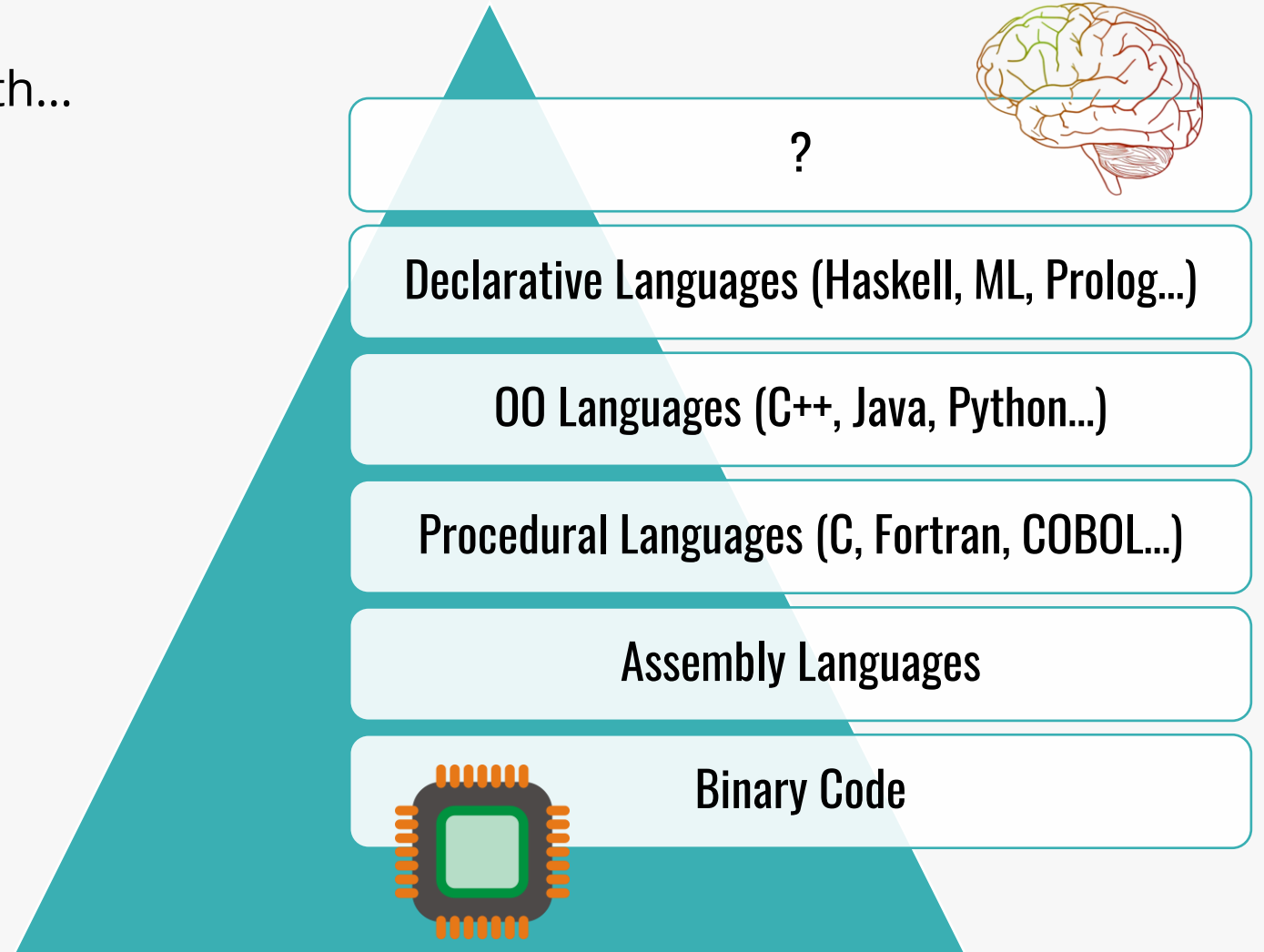
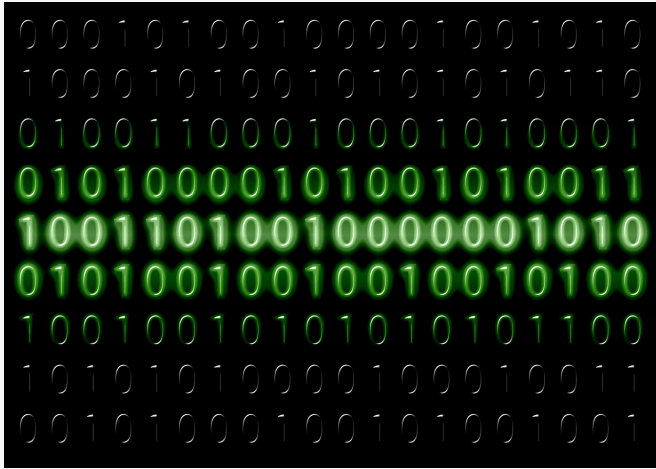
Procedural and OO Programming

03 Object Oriented Programming

...Towards a Higher Level of Abstraction

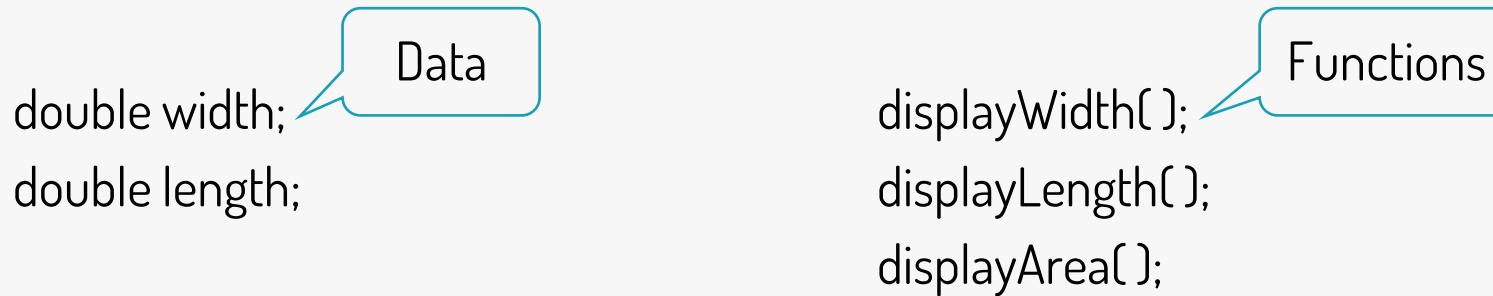
Computers just manipulate 0's and 1's

But **binary is hard (for humans)** to work with...



...Procedural Programming and its Limitations

Data is stored in a **collection of variables and/or structures**,
along with a **set of functions** that perform **operations on the data**
The data and the functions **are separate entities**



Limitations

- Variables and data structures are **passed to the functions** that perform the desired operations
- What if the data types change? **Many functions must also be changed** – **Error prone**
As the procedural programs become **larger**, function hierarchies become **more complex**:
 - difficult to understand and maintain
 - difficult to modify and extend

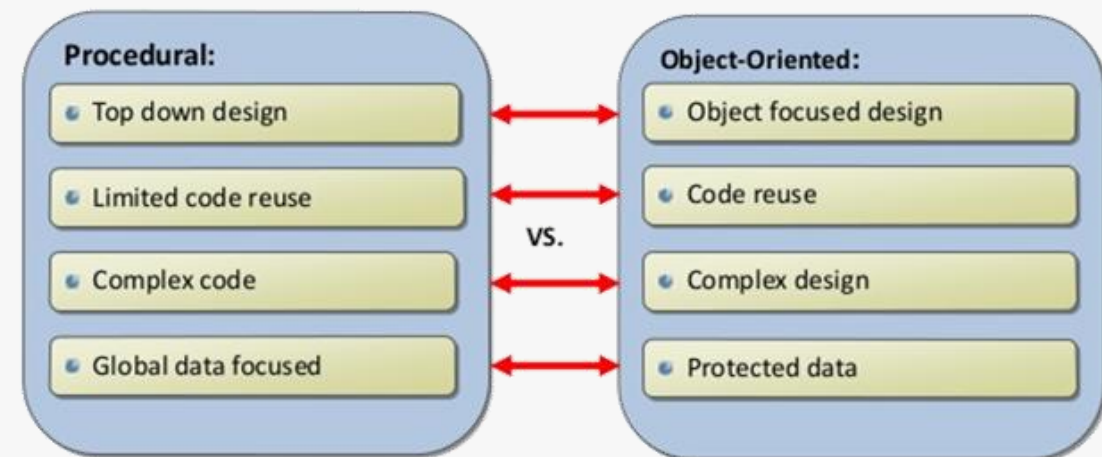
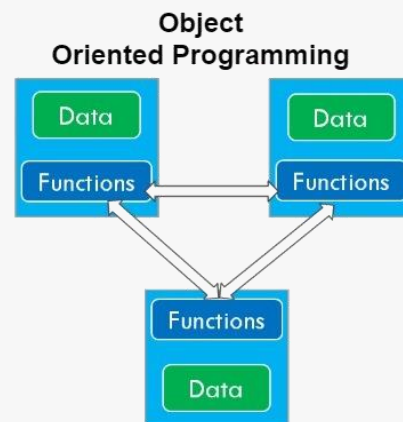
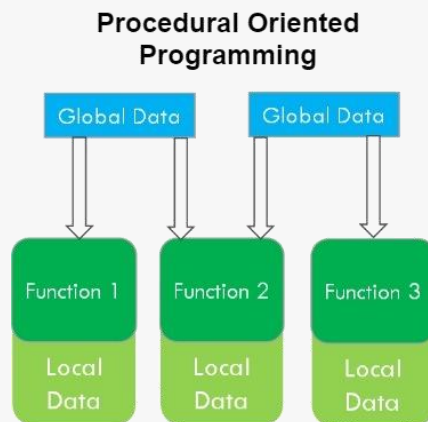
Procedural vs. Object Oriented Programming

Procedural programming focuses on the process/actions that occur in a program

Object-Oriented programming is based on the data and the functions that operate on it.

Objects are instances of ADTs that represent the data and its functions

Procedural programming focuses on **creating procedures**,
object-oriented programming focuses on **creating objects**



Object Oriented Programming – OOP

Object-oriented programming approach organizes programs in a way that mirrors the real world, in which all objects are associated with both attributes and behaviors

Object-oriented programming involves thinking in terms of objects

An OOP program can be viewed as a collection of cooperating objects



Light Bulb

- **Attributes:**
 - on (true/false)
- **Behavior**
 - Switch on
 - Switch off
 - Check if on



Bank Account

- **Attributes:**
 - balance
- **Behavior**
 - Deposit
 - Withdraw
 - Check balance



Car

- **Attributes:**
 - Gas in tank
 - Mileage
- **Behavior**
 - Accelerate
 - Brake
 - Load fuel
 - Check fuel



Bank

Deduct Zakat



Bank Account

Get Loan

Objects Interact with Each Other



Customer

Deposit
Withdraw

Object Oriented Programming – Terminology

Class: an abstract data-type or a user-defined data type (bundling of related variables)

Object: an instance of a class

A class is like a blueprint and objects are like houses built from the blueprint

An object is an instance (realization) of a class

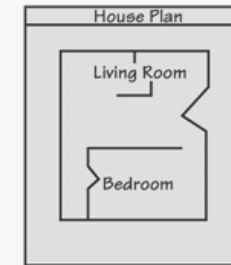
A single class can have multiple instances

Class is a Type

You can use primitive data types to define variables.

You can also use class names to declare object names.

Blueprint that describes a house.



Instances of the house described by the blueprint.



Object Oriented Programming – Terminology

Classes

Classes are constructs/templates that define objects of the same type.

A **class** uses **Variables** (data fields) to define **state**

A **class** uses **Functions** to define **behaviors**.

Additionally, a class provides a **special type of function**, known as **constructors**:

Invoked to **construct objects** from the class.

Objects

An **object** has a **unique identity**, **state**, and **behaviors**.

The **state** of an object consists of **a set of data fields** (also known as **properties**) with their current values.

The **behavior** of an object is defined by **a set of functions**.



03-02

Classes & Objects in OOP

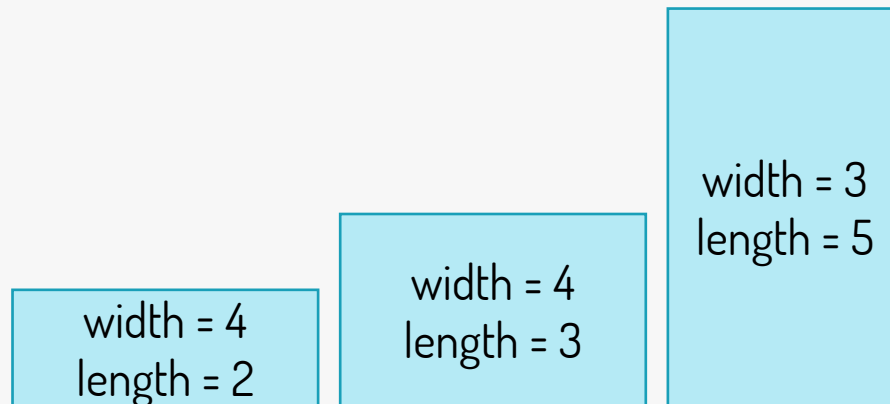
03 Object Oriented Programming

Object Oriented Programming – Classes

Objects are created from a **class**

Format:

```
class ClassName {  
    declaration;  
    declaration;  
};
```



Example

```
class Rectangle {
```

```
    double width;  
    double length;
```

Data (Attributes)

```
    displayWidth( );  
    displayLength( );  
    displayArea( );
```

Functions

```
};
```

Attribute values define the **state in objects**

Functions define the **behavior of objects**

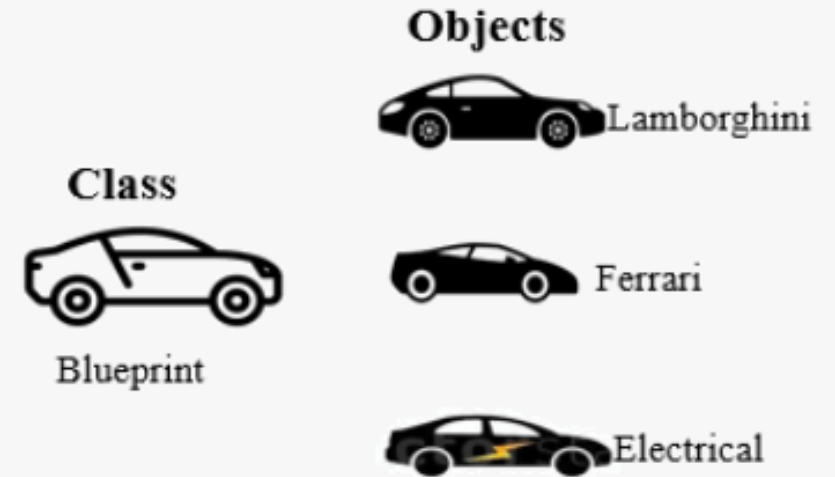
Object Oriented Programming – Objects

An object is an instance of a class, to create an object:

```
ClassName objectName;
```

```
Rectangle r;
```

Every object has a unique identity, its own state



Object Oriented Programming – Access Specifiers

Used to **control access** to members of the class

public:

can be accessed by functions
outside of the class

private:

can only be called by or accessed by functions
that are **members of the class**

Example

```
class Rectangle {
```

private:

```
double width;  
double length;
```

Private
Members

public:

```
displayWidth( );  
displayLength( );  
displayArea( );
```

Public
Members

```
};
```

Rule of Thumb:
Keep data attributes **private**
and functions **public**

To protect data from **unwanted access and modification**
a.k.a **data hiding**

Object Oriented Programming – Access Specifiers

Can be listed in **any order** in a class

Can appear **multiple times** in a class

If not specified, the default is **private**

A **class** is like **struct**, but **not the same**

Members of a **struct** are **public** by default

Members of a **class** are **private** by default

```
class Rectangle {
```

```
    double width;
```

```
    double length;
```

```
};
```

```
Rectangle r1; //class object
```

```
cout << r1.width; //ERROR, width is private
```

Correct Example

```
class Rectangle {
```

```
    private:
```

```
        double width;
```

```
        double length;
```

```
    public:
```

```
        getWidth() { return width; }
```

```
};
```

```
Rectangle r1; //class object
```

```
cout << r1.getWidth(); //works getWidth() is public
```

Object Oriented Programming – Member Functions

Within the class declaration (in-line member function)

```
class Rectangle
{
    private:
        double width;
        double length;
    public:
        double calcArea() {
            return width * length; //inline function
        }
};
```

Outside the class (out-of-line member functions)

Put prototype in class declaration

Define function using class name and scope resolution operator (::)

Combine class name with member function name

```
returnType ClassName::MemberFunctionName() {
    .....
}
```

Different classes can have member functions with the same name

```
int Rectangle::setWidth(double w) {
    width = w;
}
```

... Object Oriented Programming – Out-of-Line Member Functions

```
class Rectangle
```

```
{
```

```
    private:
```

```
        double width;
```

```
        double length;
```

```
    public:
```

```
        double calcArea() //prototype
```

```
}; //class declaration ends
```

Whether member function is inline or out-of-line
Access remains the same as declared in the class

```
double Rectangle::calcArea(){
```

```
    return width * length; //out-of-line function
```

```
}
```

Object Oriented Programming – Private Member Functions

Only accessible (callable) from
member functions of the class

No direct access possible
(with object instance of the class)

Can be: inline / out-of-line

A **private** member function can only be called
by another member function

It is used for internal processing by the class,
not for use outside of the class

```
class Rectangle {
```

```
    private:
```

```
    double width;
```

```
    double length;
```

```
    double calcArea() {  
        return width * length;
```

```
    }
```

```
    public:
```

```
    void displayArea(){  
        cout<< calcArea();  
    }
```

```
}; //class declaration ends
```

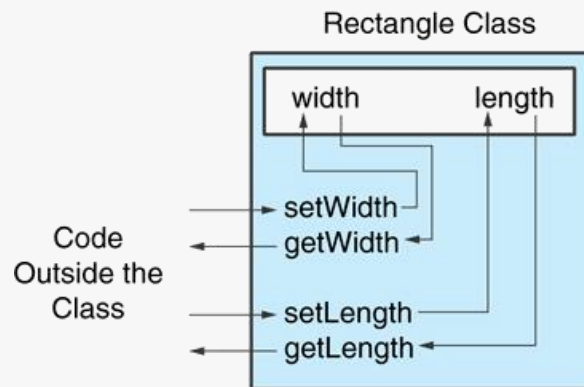
private member
function (inline)

Object Oriented Programming – Setters and Getters

Setter (Mutator): A member function that **stores or changes a value** in a private member variable

Getter (Accessor): A member function that **retrieves a value** from a private member variable.

Code outside the class must use the **class's public member functions** to interact with the object



Constraints can be added in setters to **prevent unwanted values** in data members

class Rectangle {

private:

double width;

double length;

public:

void setWidth(double w) { **width = w;** }

void setLength(double l) { **length = l;** }

double getWidth() { **return width;** }

double getLength() { **return length;** }

}; //class declaration ends

Setter(Mutator)

Getter(Accessor)

Object Oriented Programming – Const Member Functions

A **const** member function is **read-only**,
cannot change the **value** of any attribute

```
class Rectangle {  
    private:  
        double width;  
    public:  
        void changeWidth() const {  
            width++; //ERROR }  
};
```

Getters do not change an object's data,
so they should be marked **const**.

```
class Rectangle {  
    private:  
        double width;  
        double length;  
    public:  
        void setWidth(double);  
        void setLength(double);  
  
        double getWidth() const { return width; }  
        double getLength() const { return length; }  
        double getArea() const { return width * length; }  
}; //class declaration ends
```

... Avoiding Stale Data

Some data is the result of a calculation.

In the `Rectangle` class the area of a rectangle is calculated using `length x width`

If we were to use an `area` variable here in the `Rectangle` class, its value would be dependent on the length and the width.

If we change `length` or `width` without updating `area`, then `area` would become `stale`.

To avoid stale data, it is best to calculate the value of that data within a member function rather than store it in a variable.



03-03

Reference and Pointer Objects

03 Object Oriented Programming

...Pointer to an Object

Can define a pointer to an object:

```
Rectangle *rPtr;
```

Can access public members via pointer:

```
rPtr = &otherRectangle;
```

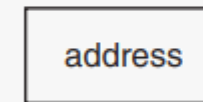
You can use `*` and `.` OR `->`

```
rPtr->setLength(12.5);
```

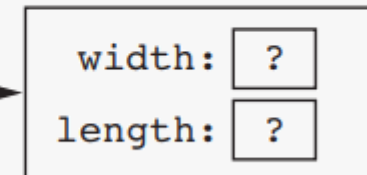
```
rPtr->setWidth(4.8);
```

```
cout << rPtr->getLength( ) << endl;
```

The `rectPtr` pointer variable
holds the address of the
`myRectangle` object



The `myRectangle` object



...Dynamically Allocating an Object

We can also use a pointer to **dynamically allocate** an object.

```
Rectangle *rPtr = new Rectangle;
```

```
rPtr->setLength(12.5);
```

```
rPtr->setWidth(10.3);
```

Deallocate memory and delete object

```
delete rPtr;
```

```
rPtr = NULL;
```

...Reference to Objects

```
// class Count definition
class Count
{
public: // public data is dangerous
    // sets the value of private data member x
    void setX( int value )
    {
        x = value;
    } // end function setX

    // prints the value of private data member x
    void print()
    {
        cout << x << endl;
    } // end function print

private:
    int x;
}; // end class Count
```

```
int main()
{
    Count counter; // create counter object
    Count *counterPtr = &counter; // create pointer to counter
    Count &counterRef = counter; // create reference to counter

    cout << "Set x to 1 and print using the object's name: ";
    counter.setX( 1 ); // set data member x to 1
    counter.print(); // call member function print

    cout << "Set x to 2 and print using a reference to an object: ";
    counterRef.setX( 2 ); // set data member x to 2
    counterRef.print(); // call member function print

    cout << "Set x to 3 and print using a pointer to an object: ";
    counterPtr->setX( 3 ); // set data member x to 3
    counterPtr->print(); // call member function print
} // end main
```

```
Set x to 1 and print using the object's name: 1
Set x to 2 and print using a reference to an object: 2
Set x to 3 and print using a pointer to an object: 3
```

...Reference and Pointers to Objects

```
class Rectangle {
```

```
    private:
```

```
    int w;           int h;
```

```
    public:
```

```
    Rectangle ( ) { }
```

```
    void setWidth(int ww) { w = ww; }
```

```
    void setHeight(int hh) { h = hh; }
```

```
    double getArea( ) const { return w * h; }
```

```
}; //class declaration ends
```

```
Area (object) = 20
```

```
Area (pointer) = 20
```

```
Area (reference to object) = 20
```

```
Area (ref to pointer) = 20
```

```
int main( ) {
```

```
    Rectangle r1;
```

```
    Rectangle *ptr = &r1;
```

```
    Rectangle &ref = r1;
```

```
    Rectangle* &ref2 = ptr;
```

```
    r1.setWidth( 5 );
```

```
    r1.setHeight( 4 );
```

```
    cout<<"\n Area (object) = "<<r1.getArea( );
```

```
    cout<<"\n Area (pointer) = "<<ptr->getArea( );
```

```
    cout<<"\n Area (reference to object) = "<<ref.getArea( );
```

```
    cout<<"\n Area (ref to pointer) = "<<ref->getArea( );
```

```
    return 0;
```

```
}
```

... Separating Specification from Implementation

Makes it easier to **modify programs**

Header files (**.h**): Contains **class definitions and function prototypes**

Source-code files (**.cpp**): Contains **member function definitions**

Place class declaration in a header file that serves as the class specification file.

Name the file **ClassName.h**, for example, **Rectangle.h**

Place member function definitions in **ClassName.cpp**, for example, **Rectangle.cpp**

File should **#include** the class specification file

Programs that use the class must **#include** the class specification file,
and be compiled and linked with the member function definitions

...Time Class with Time1.h

```
1. // time1.h - Declaration of the Time Class
2. // Member functions are defined in time1.cpp
3.
4. // prevent multiple inclusions of header file
5. #ifndef TIME1_H
6. #define TIME1_H
7.
8. // Time abstract data type definition
9. class Time {
10.     public:
11.         Time(); //constructor
12.         void setTime(int, int, int); //set hour
13.         void printMiliatry(); //print
14.         void printStandard(); //print
```

Dot (.) replaced with
underscore (_) in file name.

```
15.     private:
16.         int hour; // 0 - 23
17.         int minute; // 0 - 59
18.         int second; // 0 - 59
19. };
20.
21. #endif
```

If time1.h (TIME1_H) is not defined (#ifndef) then it is loaded (#define TIME1_H). If TIME1_H is already defined, then everything up to #endif is ignored. This prevents loading a header file multiple times.

Time Class with Time1.cpp

```

22. // Member function definition for Time Class
23. #include <iostream>
24. using std::cout
25. #include "time1.h"
26. // Constructor initializes each data member to zero
27. // Ensures all Time objects start in a consistent state.
28. Time::Time() { hour = minute = second = 0; }
29.
30. // Set time value. Perform validity checks
31. void Time::setTime( int h, int m, int s ) {
32.     hour = ( h >= 0 && h < 24 ) ? h : 0;
33.     minute = ( m >= 0 && m < 60 ) ? m : 0;
34.     second = ( s >= 0 && s < 60 ) ? s : 0;
35. }

```

Source file uses #include to load the header file

```

36. // Print time in military formal
37. void Time::printMilitary() {
38.     cout << ( hour < 10 ? "0" : " " ) << hour << " : "
39.     << ( minute < 10 ? "0" : " " ) << minute ;
40. }
41.
42. // Print time in military formal
43. void Time::printStandard() {
44.     cout << ((hour == 0 || hour == 12) ? 12 : hour % 12) << " : "
45.     << ( minute < 10 ? "0" : " " ) << minute ;
46.     << ( second < 10 ? "0" : " " ) << second
47.     << ( hour < 12 ? "AM" : "PM" ) ;
48. }

```

Source file contains function definitions



03-04

Constructors / Destructors

03 Object Oriented Programming

...Constructors

Member function that is **automatically called** when an object is created

Purpose is to **construct an object**

Constructor function name is **class name**

Has **no return type**

A class can have **multiple constructors**

...Constructors

Constructor (in-line member function)

```
class Demo {  
    public:  
    Demo() { // inline function - constructor  
        cout<<"Welcome to the constructor";  
    }  
};
```

Whenever an instance of a class is created,
the constructor is **automatically called**

```
Demo dem; // object dem is created
```

```
// Prints
```

```
Welcome to the constructor
```

Constructor (out-of-line member functions)

```
class Demo {  
    public:  
    Demo(); // constructor  
};  
  
Demo::Demo() { // out of line function - constructor  
    cout<<"Welcome to the constructor";  
}
```

... Rectangle Class – Constructor

```
class Rectangle {
```

```
    private:
```

```
    double width;
```

```
    double length;
```

```
    public:
```

```
    Rectangle(); //constructor
```

```
    void setWidth(double);
```

```
    void setLength(double);
```

```
    double getWidth() const { return width; }
```

```
    double getLength() const { return length; }
```

```
    double getArea() const { return width * length; }
```

```
}; //class declaration ends
```

```
Rectangle:: Rectangle() {
```

```
    width = 0.0;
```

```
    length = 0.0;
```

```
}
```

...What Constructors Do?

Help in **initializing** class data members

```
Employee() { id = 0; }
```

Allocate memory for dynamic members

```
Employee() {  
    char* nameptr = new char[20];  
}
```

Allocate any **needed resources**

Such as to open files, etc

... Default Constructors

A **default constructor** is a constructor that **takes no arguments**.

If you write a class with **no constructor** at all,

C++ will write a default constructor for you, one **that does nothing**.

A simple instantiation of a class (with no arguments) calls the default constructor:

Rectangle r;

A class can have **only one** default constructor

...Parametrized Constructors

To create a **constructor that takes arguments**:
indicate parameters in prototype:

```
Rectangle(double, double);
```

Use parameters in the definition:

```
Rectangle::Rectangle(double w, double len) {  
    width = w;  
    length = len;  
}
```

You can pass arguments to the constructor when you create an object:

```
Rectangle r(10.7, 5.2);
```

Constructors can be **overloaded**

```
Rectangle(double w, double len, double ar) {  
    width = w;  
    length = len;  
    area = ar;  
}
```

```
Rectangle r(10.0, 5.0, 50.0);
```

... More About Default Constructors

If **all a constructor's parameters** have default arguments, then it is a **default constructor**. e.g.:

```
Rectangle (double len = 0, double wid = 0){  
    length = len;  
    width = wid;  
}
```

Cannot create **multiple default constructors** - **ERROR**

Creating an object and **passing no arguments** will cause this constructor to execute:

Rectangle r ;

Rectangle **r1** (10.0);

Rectangle **r2** (10.0, 2.0);

All three cause the **same constructor to execute**

With this constructor, cannot create any other constructor with 0,1 or 2 parameters. Because then the **compiler cannot differentiate** and decide which one to call

Classes with No Default Constructor

When **all class's constructors require arguments**, then the class has **NO default constructor**.

When this is the case, you **must pass the required arguments** to the constructor when creating an object.

••• Destructors

Member function **automatically** called when an **object is destroyed**

Destructor name is **~classname**, *e.g.*, **~Rectangle**

Has **no return type**; takes **no arguments**

Only one destructor per class, *i.e.*, it **cannot be overloaded**

If constructor allocates dynamic memory, destructor should release it

Inventory Class – Example

Contents of InventoryItem.h (Version 1)

```

1 // Specification file for the InventoryItem class.
2 #ifndef INVENTORYITEM_H
3 #define INVENTORYITEM_H
4 #include <cstring> // Needed for strlen and strcpy
5
6 // InventoryItem class declaration.
7 class InventoryItem
8 {
9 private:
10     char *description; // The item description
11     double cost;       // The item cost
12     int units;         // Number of units on hand
13 public:
14     // Constructor
15     InventoryItem(char *desc, double c, int u)
16     { // Allocate just enough memory for the description.
17         description = new char [strlen(desc) + 1];
18
19         // Copy the description to the allocated memory.
20         strcpy(description, desc);
21
22         // Assign values to cost and units.
23         cost = c;
24         units = u;}
25
26     // Destructor
27     ~InventoryItem()
28     { delete [] description; }
29

```

Program 13-11

```

1 // This program demonstrates a class with a destructor.
2 #include <iostream>
3 #include <iomanip>
4 #include "InventoryItem.h"
5 using namespace std;
6
7 int main()
8 {
9     // Define an InventoryItem object with the following data:
10    // Description: Wrench Cost: 8.75 Units on hand: 20
11    InventoryItem stock("Wrench", 8.75, 20);
12
13    // Set numeric output formatting.
14    cout << setprecision(2) << fixed << showpoint;
15
16    // Display the object's data.
17    cout << "Item Description: " << stock.getDescription() << endl;
18    cout << "Cost: $" << stock.getCost() << endl;
19    cout << "Units on hand: " << stock.getUnits() << endl;
20    return 0;
21 }

```

Program Output

```

Item Description: Wrench
Cost: $8.75
Units on hand: 20

```

••• Constructors, Destructors, and Dynamically Allocated Objects

When an object is **dynamically allocated** with the new operator, its constructor executes:

```
Rectangle *r = new Rectangle(10, 20);
```

When the object is **destroyed**, its destructor executes:

```
delete r;
```

... Only One Default Constructor and One Destructor

Do not provide more than one default constructor for a class: one that takes no arguments and one that has default arguments for all parameters

```
Square();
```

```
Square(int = 0); // will not compile
```

Since a destructor takes no arguments,
there can only be **one destructor for a class**

... Member Function Overloading

Non-constructor member functions can also be overloaded:

```
void setCost(double);
```

```
void setCost(char *);
```

Must have **unique parameter lists**, like overloaded constructors

...When Constructors and Destructors Are Called?

Constructors and destructors called **automatically**

1. **Global scope objects**

Constructors called before any other function (including main)

Destructors called when main terminates (or exit function called)

2. **Local scope objects**

Constructors called when objects are defined

Destructors called when objects leave scope

... Create and Destroy

```

1.  class CreateAndDestroy {
2.      public:
3.          CreateAndDestroy( int ); // constructor
4.          ~CreateAndDestroy( ); // destructor
5.  };
6.  CreateAndDestroy::CreateAndDestroy( int value ) {
7.      data = value;
8.      cout << "Object " << data << " constructor";
9.  }
10. CreateAndDestroy::~~CreateAndDestroy( ) {
11.     cout << "Object " << data << " destructor " << endl;
12. }

```

Constructor and Destructor changed to print when they are called.

```

Object 1 constructor (global created before main)
Object 2 constructor (local in main)
Object 5 constructor (local in create)
Object 7 constructor (local in create)
Object 7 destructor
Object 5 destructor
Object 4 constructor (local in main)
Object 4 destructor
Object 2 destructor
Object 1 destructor

```

```

15. void create( void ); // prototype
16. CreateAndDestroy first( 1 ); // global object
17. int main( ) {
18.     cout << "(global created before main)" << endl;
19.     CreateAndDestroy second( 2 ); // local object
20.     cout << " (local in main)" << endl;
21.     create(); // call function to create objects
22.     CreateAndDestroy fourth( 4 ); // local object
23.     cout << " (local in main)" << endl;
24.     return 0;
25. }
26. // Function to create objects
27. void create( void ) {
28.     CreateAndDestroy fifth( 5 );
29.     cout << " (local in create)" << endl;
30.     CreateAndDestroy seventh ( 7 );
31.     cout << " (local in create)" << endl;
32. }

```

... Array of Pointers to Objects

Declare an **array of pointer to objects**

Allocate and initialize each object in a loop

```
Date *dates[31];  
for (int day = 0; day < 31; ++day) {  
    dates[day] = new Date(3, day, 2024);  
}
```



03-05

Copy Constructors

03 Object Oriented Programming

... Default Member-wise Assignment

Assignment operator (=) can be used to **assign** an object to another object of the **same type**.

Member-wise assignment:

each data member of the object on the right of the assignment operator is assigned individually to the same data member in the object on the left

box2

length = 10.0
width = 20.0

box1

length = 10.0
width = 20.0

```
class Rectangle {
```

```
    double length;
```

```
    double width;
```

```
    public:
```

```
    Rectangle(){ //default constructor
```

```
        length = 0;        width = 0; }
```

```
    //parametrized constructor
```

```
    Rectangle(double l, double w) {
```

```
        length = l;        width = w; }
```

```
};
```

```
//box1 is an object of class Rectangle
```

```
Rectangle box1 (10.0, 20.0);
```

```
Rectangle box2;
```

```
box2 = box1;
```

... Default Member-wise Assignment

```
class Date
{
public:
    Date( int = 1, int = 1, int = 2000 ); // defa
    void print();
private:
    int month;
    int day;
    int year;
}; // end class Date

// Date constructor (should do range checking)
Date::Date( int m, int d, int y )
{
    month = m;
    day = d;
    year = y;
} // end constructor Date

// print Date in the format mm/dd/yyyy
void Date::print()
{
    cout << month << '/' << day << '/' << year;
} // end function print
```

```
int main()
{
    Date date1( 7, 4, 2004 );
    Date date2; // date2 defaults to 1/1/2000

    cout << "date1 = ";
    date1.print();
    cout << "\ndate2 = ";
    date2.print();

    date2 = date1; // default memberwise assignment

    cout << "\n\nAfter default memberwise assignment, date2 = ";
    date2.print();
    cout << endl;
} // end main
```

```
date1 = 7/4/2004
date2 = 1/1/2000

After default memberwise assignment, date2 = 7/4/2004
```

... Default Copy Constructor

A **copy constructor** creates an object and initializes it with **another object's data**.

Objects must be of the **same type**

If a class doesn't have a copy constructor, C++ creates a **default copy constructor** for it.

```
Rectangle box3 = box2;
```

OR

```
Rectangle box3(box2);
```

The **default copy constructor** performs the **member-wise assignment** when an object is initialized using another object.

...Copy Constructor

A type of constructor that is used to **initialize an object** with **another object of the same type** is known as **copy constructor**.

It is by default available in all classes

It has the same form as other constructors, except it has a **reference parameter** of the **same class type**

```
ClassName(ClassName &Variable)
```

```
Rectangle(Rectangle &r)
```

Copy Constructor for Class Date

```
Date::Date(Date &date) {  
    month = date.month;  
    day  = date.day;  
    year = date.year;  
}
```

Private members can be accessed with a **.** here because the **copy constructor is a member of the Date class**

Implicitly called in 3 situations:

1. Defining a **new object** from an **existing object**
2. **passing** an object by value
3. **returning** an object by value

... Copy Constructor

1. Defining a New Object

```
//parametrized constructor called
```

```
Date d1(2,28,2020);
```

```
// initialize 2 local objects from d1
```

```
Date d2(d1);
```

```
Date d3 = d1;
```

```
// init a dynamic object from d1
```

```
Date* pdate = new Date(d1);
```

2. Passing Objects by Value

```
//copy constructor called for each value arg
```

```
int dateDiff(Date d1, Date d2);
```

```
...
```

```
Date today; //default constructor called
```

```
Date d3(02, 21, 2000); //parametrized constr.
```

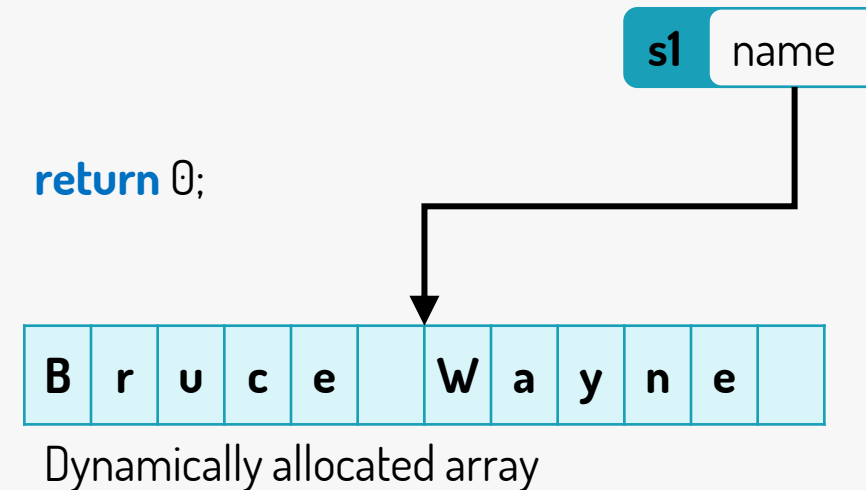
```
cout << dateDiff(d3, today);
```

...Some Issues with the Default Copy Constructor

There are instances, where the default copy constructor **can be problematic**

```
class Student {  
    private:  
    char *name;  
    public:  
    Student() { //default constructor  
        name = new char[12]{0};  
    }  
    void setName(char* nameVal) { //setter for name  
        strcpy(name, nameVal);  
    }  
    char* getName() { //getter for name  
        return name;  
    }  
};
```

```
int main() {  
    Student s1;  
    char name1[] = "Bruce Wayne";  
  
    return 0;  
}
```



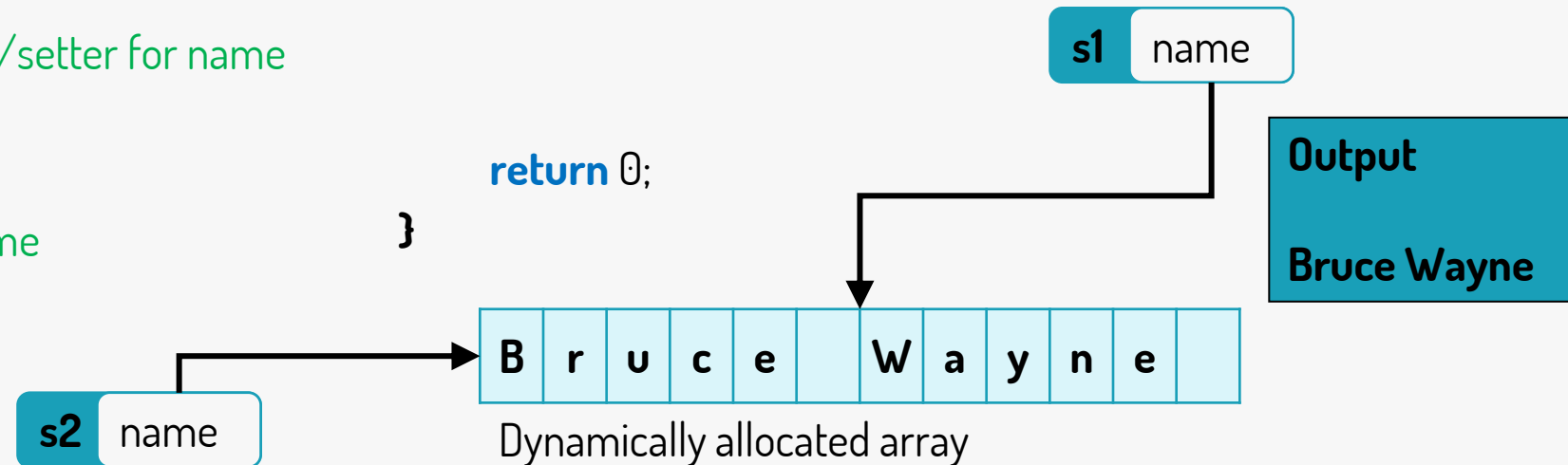
...Some Issues with the Default Copy Constructor

There are instances, where the default copy constructor **can be problematic**

```
class Student {  
    private:  
    char *name;  
    public:  
    Student() { //default constructor  
        name = new char[12]{0};  
    }  
    void setName(char* nameVal) { //setter for name  
        strcpy(name, nameVal);  
    }  
    char* getName() { //getter for name  
        return name;  
    }  
};
```

```
int main() {  
    Student s1;  
    char name1[] = "Bruce Wayne";  
    s1.setName(name1);  
    cout << s1.getName() << endl;  
    Student s2 = s1;  
}
```

s2's **default copy constructor** is called and s1's member variable values are **copied in s2**



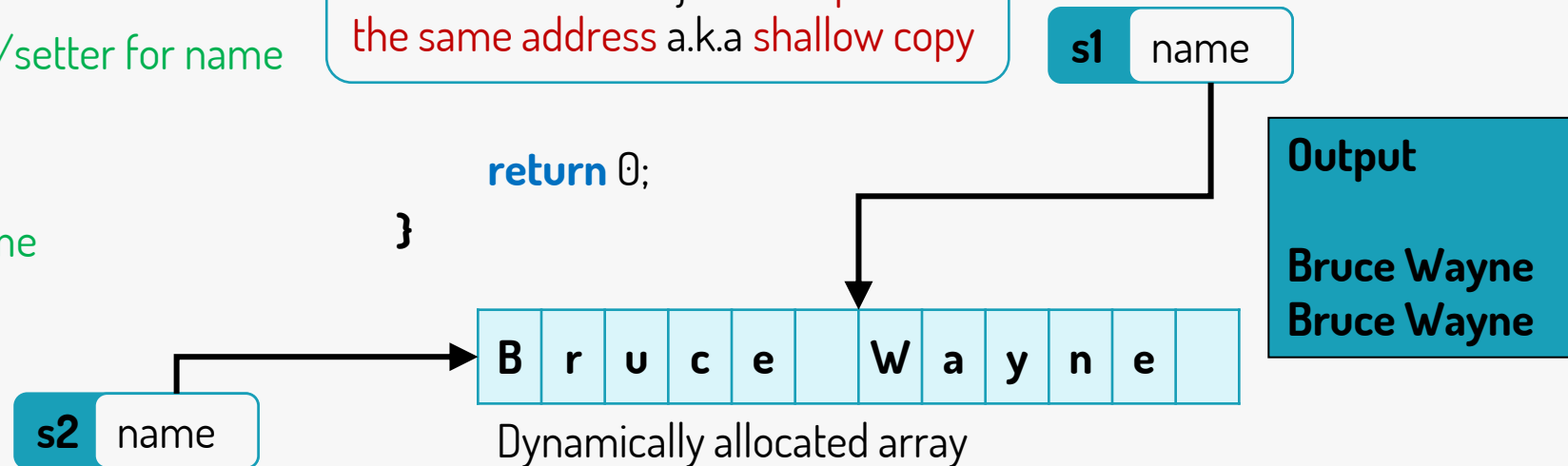
...Some Issues with the Default Copy Constructor

There are instances, where the default copy constructor **can be problematic**

```
class Student {  
    private:  
    char *name;  
    public:  
    Student() { //default constructor  
        name = new char[12]{0};  
    }  
    void setName(char* nameVal) { //setter for name  
        strcpy(name, nameVal);  
    }  
    char* getName() { //getter for name  
        return name;  
    }  
};
```

```
int main() {  
    Student s1;  
    char name1[] = "Bruce Wayne";  
    s1.setName(name1);  
    cout<<s1.getName()<<endl;  
    Student s2 = s1;  
    cout<<s2.getName()<<endl;  
}
```

Pointers in both objects will **point to the same address** a.k.a **shallow copy**

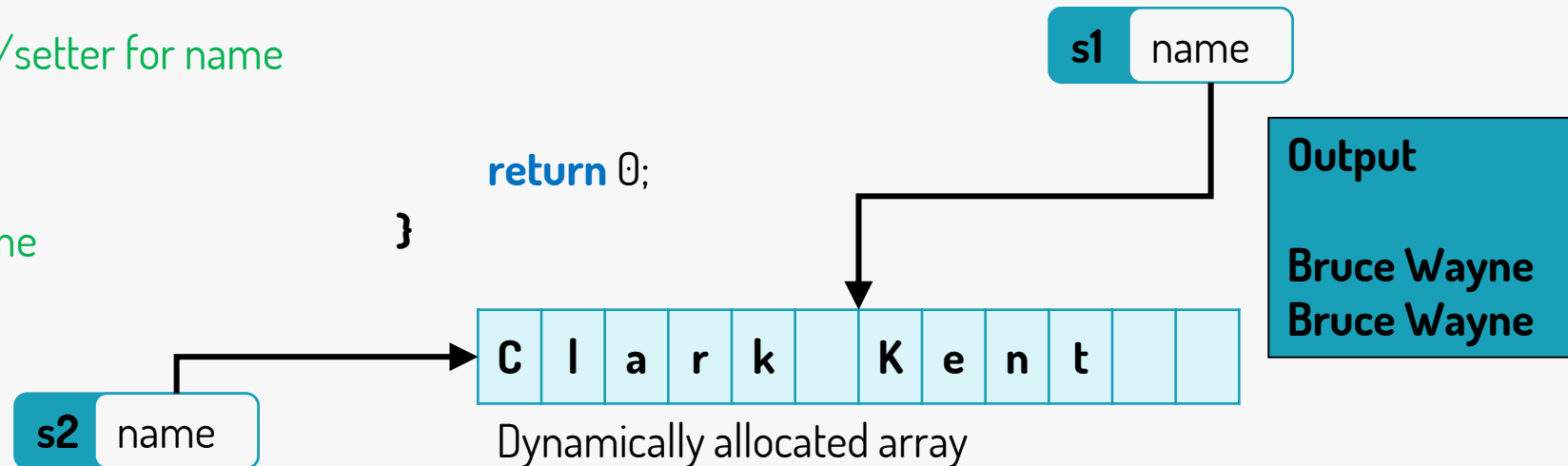


...Some Issues with the Default Copy Constructor

There are instances, where the default copy constructor **can be problematic**

```
class Student {
    private:
    char *name;
    public:
    Student() { //default constructor
        name = new char[12]{0};
    }
    void setName(char* nameVal) { //setter for name
        strcpy(name, nameVal);
    }
    char* getName() { //getter for name
        return name;
    }
};
```

```
int main() {
    Student s1;
    char name1[] = "Bruce Wayne";
    s1.setName(name1);
    cout << s1.getName() << endl;
    Student s2 = s1;
    cout << s2.getName() << endl;
    char name2[] = "Clark Kentt";
    s2.setName(name2);
    return 0;
}
```

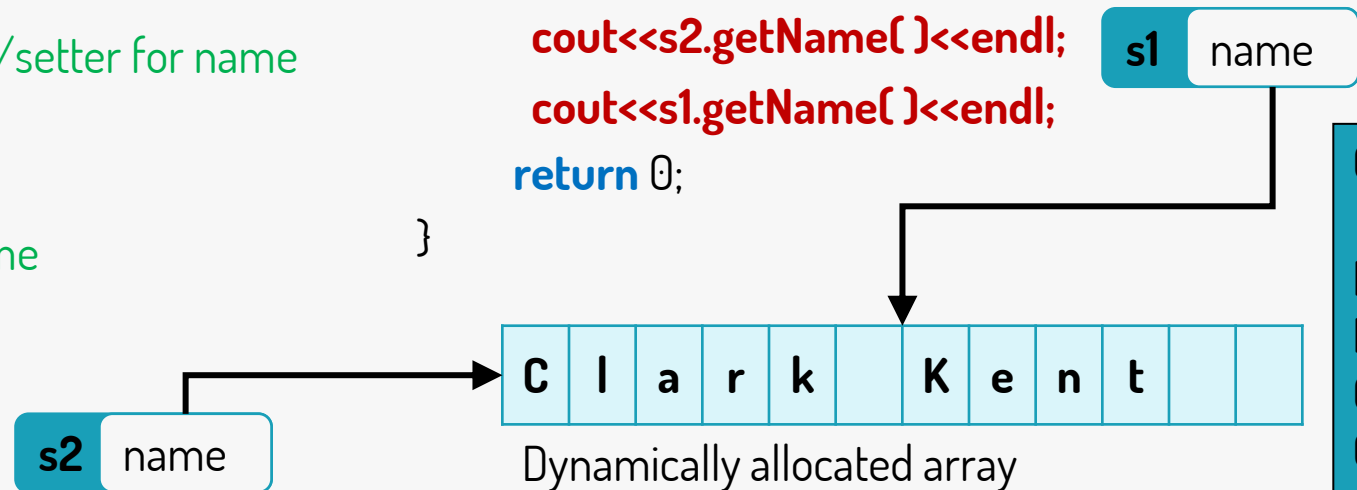


...Some Issues with the Default Copy Constructor

There are instances, where the default copy constructor **can be problematic**

```
class Student {
    private:
    char *name;
    public:
    Student() { //default constructor
        name = new char[12]{0};
    }
    void setName(char* nameVal) { //setter for name
        strcpy(name, nameVal);
    }
    char* getName() { //getter for name
        return name;
    }
};
```

```
int main() {
    Student s1;
    char name1[] = "Bruce Wayne";
    s1.setName(name1);
    cout<<s1.getName()<<endl;
    Student s2 = s1;
    cout<<s2.getName()<<endl;
    char name2[] = "Clark Kentt";
    s2.setName(name2);
    cout<<s2.getName()<<endl;
    cout<<s1.getName()<<endl;
    return 0;
}
```



Output

```
Bruce Wayne
Bruce Wayne
Clark Kentt
Clark Kentt
```

...Some Issues with the Default Copy Constructor

Either object can **manipulate the values stored in the array**, causing the changes to show up in the other object.

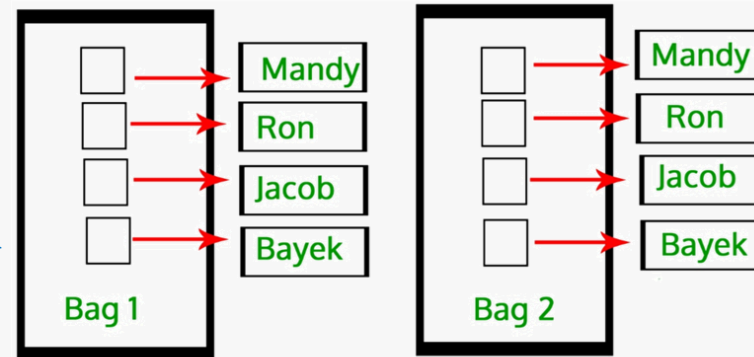
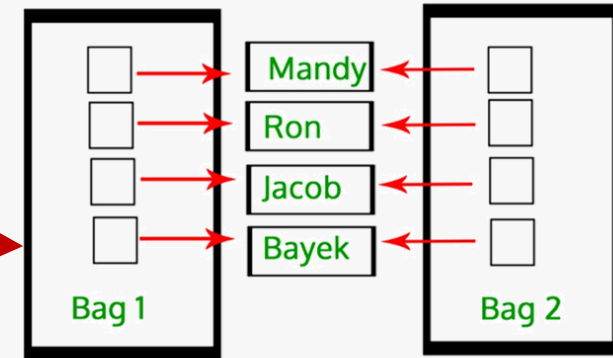
One object can be destroyed, causing its destructor to be called, which **frees the allocated memory**

The remaining object's name pointer would **still reference this section of memory**, although it should no longer be used

...User-Defined Copy Constructor, Required?

Default-copy Constructor do only “Shallow Copy”

We need user-defined copy-constructor,
When we need “Deep Copy” (for Dynamic Memory)



...User Defined Copy Constructor – Deep Copy

```

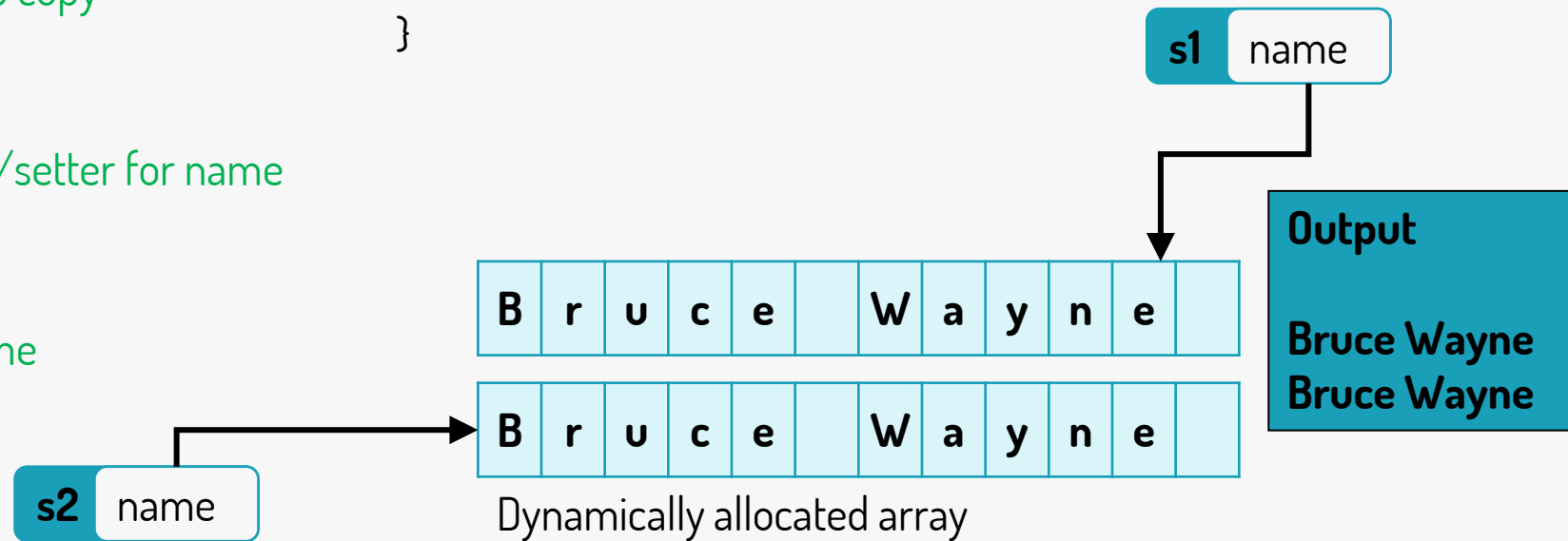
class Student {
    private:
    char *name;
    public:
    Student() { //default constructor
        name = new char[12]{0};
    }
    Student(const Student &s) { //copy constructor
        name = new char[12]; //for deep copy
        strcpy(name, s.name);
    }
    void setName(char* nameVal) { //setter for name
        strcpy(name, nameVal);
    }
    char* getName() { //getter for name
        return name;
    }
};

```

```

int main() {
    Student s1;
    char name1[] = "Bruce Wayne";
    s1.setName(name1);
    cout << s1.getName() << endl;
    Student s2 = s1;
    cout << s2.getName() << endl;
    return 0;
}

```



... User Defined Copy Constructor – Deep Copy

```

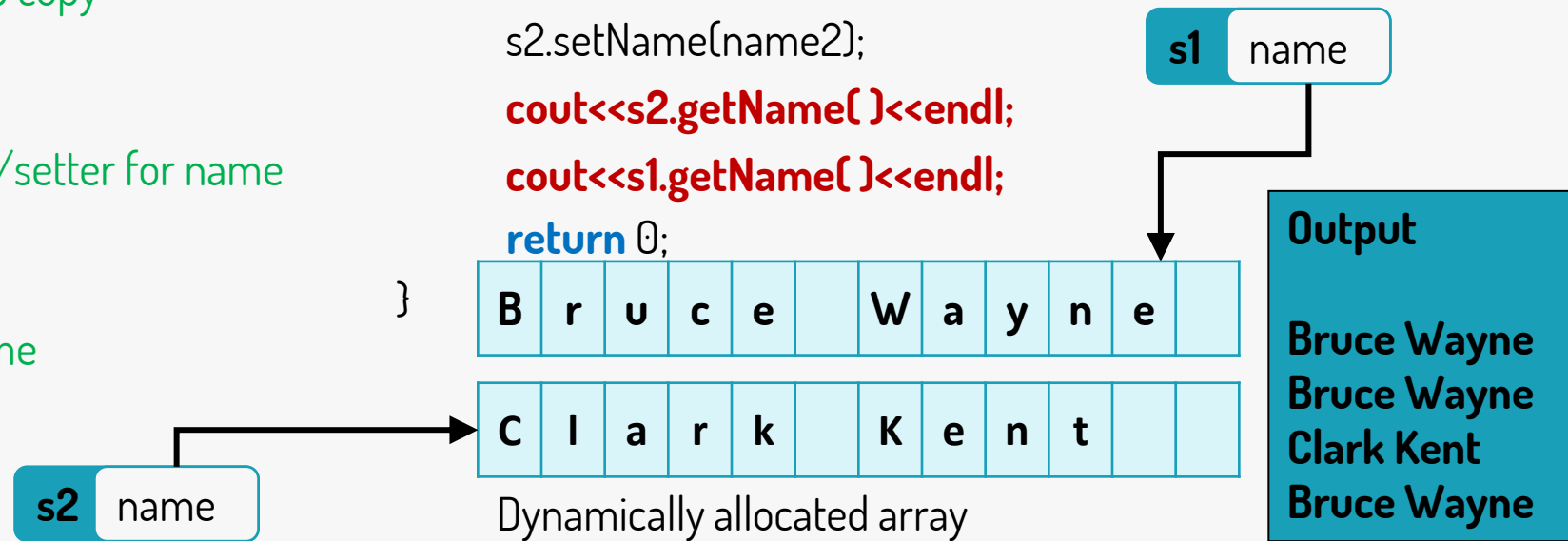
class Student {
    private:
    char *name;
    public:
    Student() { //default constructor
        name = new char[12]{0};
    }
    Student(const Student &s) { //copy constructor
        name = new char[12]; //for deep copy
        strcpy(name, s.name);
    }
    void setName(char* nameVal) { //setter for name
        strcpy(name, nameVal);
    }
    char* getName() { //getter for name
        return name;
    }
};

```

```

int main() {
    Student s1;
    char name1[] = "Bruce Wayne";
    s1.setName(name1);
    cout<<s1.getName()<<endl;
    Student s2 = s1;
    cout<<s2.getName()<<endl;
    char name2[] = "Clark Kent";
    s2.setName(name2);
    cout<<s2.getName()<<endl;
    cout<<s1.getName()<<endl;
    return 0;
}

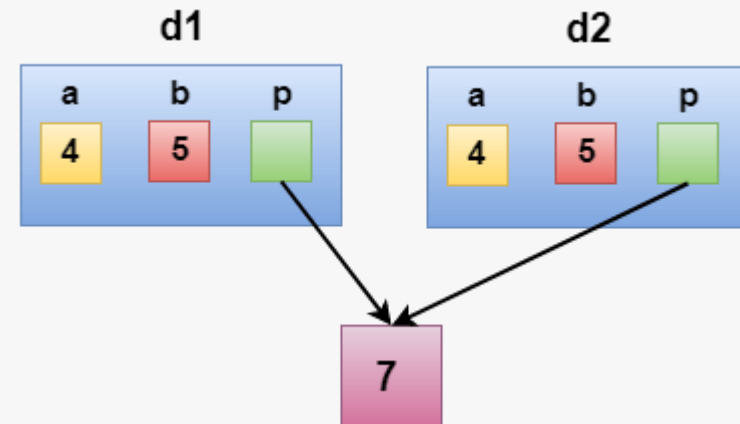
```



...Shallow Copy

```
1. class Demo {
2.     int a;
3.     int b;
4.     int *p;
5.     public:
6.     Demo() {
7.         p=new int;
8.     }
9.     void setdata(int x,int y,int z) {
10.         a=x;
11.         b=y;
12.         *p=z;
13.     }
14.     void showdata() {
15.         std::cout << "value of a is : " <<a<< std::endl;
16.         std::cout << "value of b is : " <<b<< std::endl;
17.         std::cout << "value of *p is : " <<*p<< std::endl;
18.     }
19. };
```

```
21. int main() {
22.     Demo d1;
23.     d1.setdata(4,5,7);
24.     Demo d2 = d1;
25.     d2.showdata();
26.     return 0;
27. }
```



...Shallow Copy

```

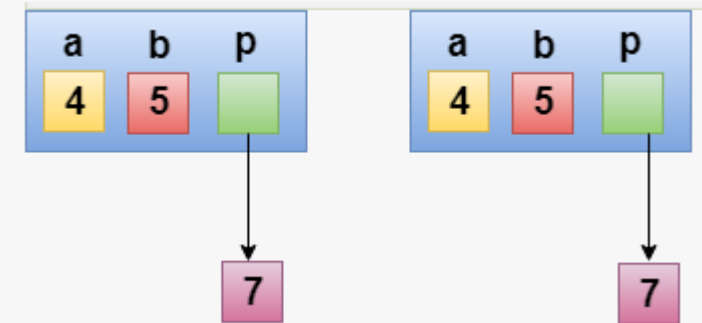
1. class Demo {
2.     int a;
3.     int b;
4.     int *p;
5. public:
6.     Demo() {
7.         p=new int;
8.     }
9.     Demo(Demo &d) {
10.        a = d.a;
11.        b = d.b;
12.        p = new int;
13.        *p = *(d.p);
14.    }
15.    void setdata(int x,int y,int z) {
16.        a=x;
17.        b=y;
18.        *p=z;
19.    }

```

```

1. void showdata() {
2.     std::cout << "value of a is : " <<a<< std::endl;
3.     std::cout << "value of b is : " <<b<< std::endl;
4.     std::cout << "value of *p is : " <<*p<< std::endl;
5. }
6. };
7.
8. int main() {
9.     Demo d1;
10.    d1.setdata(4,5,7);
11.    Demo d2 = d1;
12.    d2.showdata();
13.    return 0;
14. }

```





03-06

Static Class Variables and Functions

03 Object Oriented Programming

...Class Instance Variables (Attributes)

```
class Car {  
    int fuel;  
    string model;  
    Car() { //constructor  
        fuel = 0;  
        model = "";  
    }  
    setFuel(int f) {  
        fuel = f;  
    }  
};
```

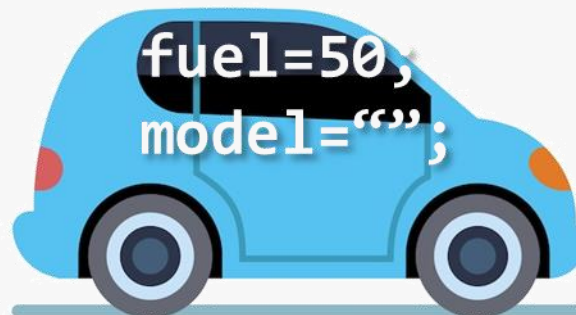
```
Car car1, car2, car3;
```

```
// class has a setter
```

```
car1.setFuel(50);
```

```
car2.setFuel(30);
```

```
car3.setFuel(84);
```



car1



car2



car3

...static Class Members

Class members include **instance variables** and functions

It is possible to create a member variable or member function that does **not belong to any instance of a class**.

Such members are known as a **static member variables** and **static member functions**.

You can think of **static class members** as **belonging to the class** instead of to an instance of the class

...static Member Variables

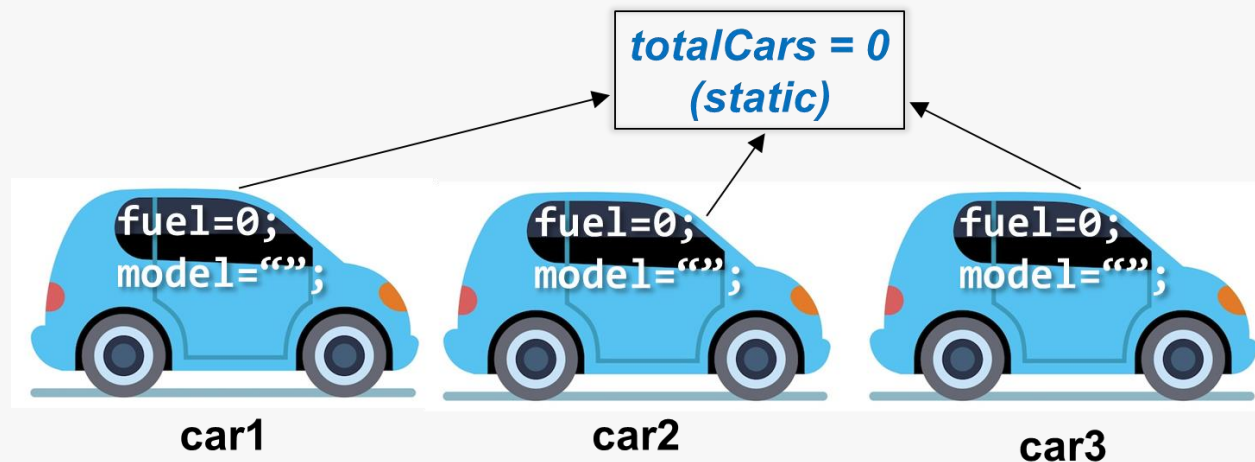
```
class Car {  
    int fuel;  
    string model;  
    static int totalCars;  
    Car() { //constructor  
        fuel = 0;  
        model = "";  
    }  
};
```

```
int Car::totalCars;  
Car car1, car2, car3;
```

When a member variable is **static**, there will be **only one copy** of it in memory, regardless of the **number of instances of the class that might exist**.

A single copy of a class's static member variable is **shared by all objects of the class**

Static member variables exist even if **no instances (objects) of the class exist**



...static Member Variables

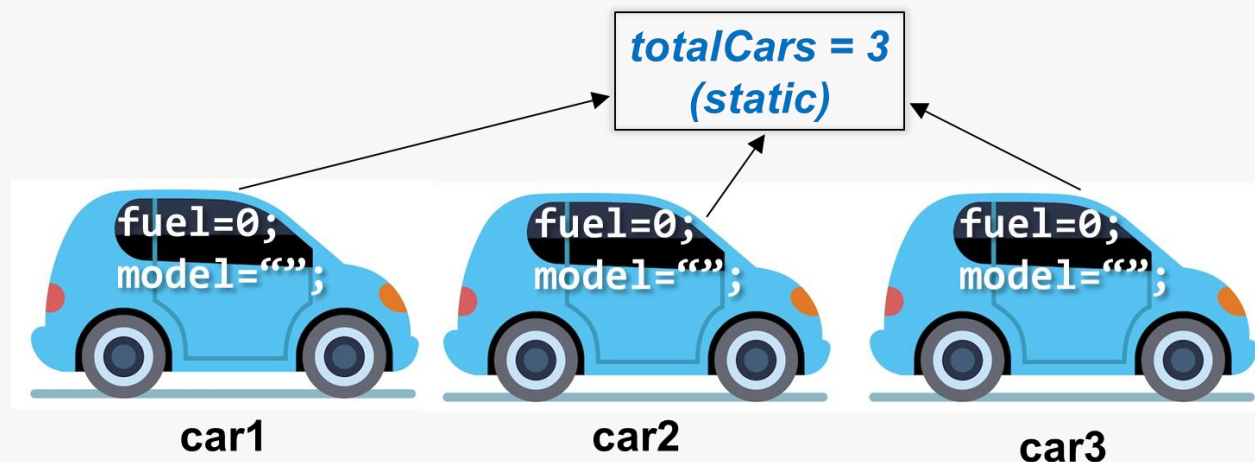
```
class Car {  
    int fuel;  
    string model;  
    static int totalCars;  
    Car() { //constructor  
        fuel = 0;  
        model = "";  
        totalCars++;  
    }  
};
```

```
int Car::totalCars;  
Car car1, car2, car3;
```

When a member variable is **static**, there will be **only one copy** of it in memory, regardless of the **number of instances of the class that might exist**.

A single copy of a class's static member variable is **shared by all objects of the class**

Static member variables exist even if **no instances (objects) of the class exist**



...static Member Variables

Shared by all objects of a class

Efficient, when a **single copy of data is enough**

Only the static variable must be updated

May seem like global variables, but **have class scope**
only **accessible to objects of same class**

Exist even if **no instances (objects) of the class exist**

Can be **public** or **private**

Two-Step Procedure:

Declare (Inside Class): `static int radius;`

Define (Outside Class): `int Circle::radius;`

Static Variables

Default Initialization: 0 or Null (for pointers)

Initialization: user defined value

Initialization is made just once, at compile time.

Accessibility: Private or Public

This external definition statement **causes the variable to be created in memory** and is required.

...static Member Variables

public static Members Variables

Can be accessed using **class name**:

```
cout << Car::totalCars;
```

Can be accessed via any **class' object**:

```
cout << car1.totalCars;
```

Can be accessed via **Non-Static member functions**:

Must create object first

```
cout << car1.getCars( );
```

Can be accessed via **Static member functions**:

```
cout << Car::getTotalCars( );
```

```
cout << car1.getTotalCars( ); //public static
```

private static Members Variables

Cannot be accessed using **class name**:

```
cout << Car::totalCars; // ERROR
```

Cannot be accessed via any **class' object**:

```
cout << car1.totalCars; // ERROR
```

Can be accessed via **Non-Static member functions**:

Must create object first

```
cout << car1.getCars( );
```

Can be accessed via **Static member functions**:

```
cout << Car::getTotalCars( );
```

```
cout << car1.getTotalCars( ); //public static
```

...static Member Variables

The lifetime of a class's static member variable is the **lifetime of the program**

This means that a class's static member variables come into existence **before any instances of the class** are created

Static member functions can operate only on **static member variables**.

You can think of static member functions as **belonging to the class** instead of to an instance of the class.

Static member functions can be called **without creating any class objects**

...static Member Functions

```
class Car {  
    int fuel;  
    string model;  
    static int totalCars;  
    Car() { //constructor  
        fuel = 0;    model = "";  
        totalCars++;  
    }  
    static int getTotalCars() {  
        return totalCars; //only access static members  
    }  
};  
  
int Car::totalCars;
```

Non-static function:

Can access: all class members

Static functions:

Can access: static data and static functions

Cannot access: non-static data, non-static functions, and **this** pointer

...static Member Functions

public static Members Functions

Can be invoked using **any class object**:

```
cout << car1.getTotalCars( );
```

Can be invoked using **class name**:

```
cout << Car::getTotalCars( );
```

private static Members Functions

Cannot be invoked using **any class object**:

```
cout << car1.getTotalCars( );           // ERROR
```

Cannot be invoked using **class name**:

```
cout << Car::getTotalCars( );           // ERROR
```

Can be invoked within class:

Static member functions

Non-static member functions

...Budget.h

```
#ifndef BUDGET_H
#define BUDGET_H
// Budget class declaration
class Budget {
    private:
        static double corpBudget; // Static member
        double divisionBudget; // Instance member
    public:
        Budget() {
            divisionBudget = 0;
        }
        void addBudget(double b) {
            divisionBudget += b;
            corpBudget += b;
        }
};

double getDivisionBudget() const {
    return divisionBudget;
}
double getCorpBudget() const {
    return corpBudget;
}

// Definition of static member variable corpBudget
double Budget::corpBudget = 0;

#endif
```

...Budget.cpp

```
int main() {
    int count; // Loop counter
    const int NUM_DIVISIONS = 4; // Number of divisions
    Budget divisions[NUM_DIVISIONS]; // Array of Budget objects

    // Get the budget requests for each division.
    for (count = 0; count < NUM_DIVISIONS; count++) {
        double budgetAmount;
        cout << "Enter the budget request for division ";
        cout << (count + 1) << ": ";
        cin >> budgetAmount;
        divisions[count].addBudget(budgetAmount);
    }

    // Display the budget requests and the corporate budget.
    cout << fixed << showpoint << setprecision(2);
    cout << "\nHere are the division budget requests:\n";
```

```
    for (count = 0; count < NUM_DIVISIONS; count++) {
        cout << "\tDivision " << (count + 1) << "\t$ ";
        cout << divisions[count].getDivisionBudget() << endl;
    }
    cout << "\tTotal Budget Requests:\t$ ";
    cout << divisions[0].getCorpBudget() << endl;

    return 0;
}
```

```
Enter the budget request for division 1: 100000 [Enter]
Enter the budget request for division 2: 200000 [Enter]
Enter the budget request for division 3: 300000 [Enter]
Enter the budget request for division 4: 400000 [Enter]
```

Here are the division budget requests:

Division 1 \$ 100000.00

Division 2 \$ 200000.00

Division 3 \$ 300000.00

Division 4 \$ 400000.00

Total Budget Requests: \$ 1000000.00





03-07 this Pointer

03 Object Oriented Programming

...this Pointer

The **this** pointer is a special built-in pointer that is available to a **class's non-static member functions**.

It always points to the **instance of the class making the function call**.

The **this** pointer is passed as a **hidden argument** to all non-static member functions.

```
void setFuel(int fuel, Car* const this) {  
    this->fuel = fuel;  
}
```

```
Car car1, car2;  
car1.setFuel(20); // this pointer points at car1  
car2.setFuel(40); // this pointer points at car2
```

this pointer is a **constant pointer** that references to the calling object.

...Using the this Pointer

Not part of the **object itself**

Can be used to access instance variables within **member functions** (including constructors and destructors)

this pointer stores the **address of the calling object**

For a member function print data member x, either

this->x;

or

(*this).x

Useful when function parameters and class member variables have the same name

//Parametrized Constructor

```
Car(int fuel, string model) {
```

```
    this->fuel = fuel;
```

```
    this->model = model;
```

```
}
```

Function returns a reference pointer to the same object

```
{ return *this; }
```

Other functions can operate on that pointer

Useful for **cascading functions** - multiple functions called on the same object

...This Pointer – Example

// Using the **this** pointer to refer to object members.

```
#include<iostream>
```

```
usingstd::cout;
```

```
usingstd::endl;
```

```
class Test {
```

```
    public:
```

```
        Test( int= 0 );    // default constructor
```

```
        voidprint( ) const;
```

```
    private:
```

```
        intx;
```

```
};
```

```
Test::Test( inta ) { x = a; } //default constructor implementation
```

```
void Test::print( ) const { // ( ) around *this required
```

```
    cout << " using variable = " << x
```

```
    << "\n using this pointer = " << this->x
```

```
    << "\n using this with * = " << ( *this).x << endl;
```

```
}
```

```
int main( ) {
```

```
    Test testObject( 12 );
```

```
    testObject.print( );
```

```
    return 0;
```

```
}
```

```
using variable    = 12
using this pointer = 12
using this with * = 12
```

...Time.h

```
#ifndef TIME_H
```

```
#define TIME_H
```

```
// Time class declaration
```

```
class Time {
```

```
private:
```

```
    Time( int= 0, int =0, int= 0 ); // default constructor
```

```
    // set functions
```

```
    Time &setTime( int, int, int); // set hour, minute, second
```

```
    Time &setHour( int); // set hour
```

```
    Time &setMinute( int); // set minute
```

```
    Time &setSecond( int); // set second
```

Notice the **Time &**-function returns a **reference to a Time object**. Specify object in function definition.

```
};
```

```
// get functions (normally declared const)
```

```
int getHour( ) const; // return hour
```

```
int getMinute( ) const; // return minute
```

```
int getSecond( ) const; // return second
```

```
// print functions (normally declared const)
```

```
void printMilitary( ) const; // print military time
```

```
void printStandard( ) const; // print standard time
```

```
private:
```

```
int hour; // 0 -23
```

```
int minute; // 0 -59
```

```
int second; // 0 -59
```

...Time.cpp

```
#include <iostream>
using std::cout;
#include "time.h"
```

```
// Constructor function to initialize private data.
// Calls member function setTime to set variables.
// Default values are 0 (see class definition).
```

```
Time::Time( int hr, int min, int sec ) { setTime( hr, min, sec ); }
```

```
// Set the values of hour, minute, and second.
```

```
Time &Time::setTime( int h, int m, int s ) {
```

```
    setHour( h );
    setMinute( m );
    setSecond( s );
```

```
    return *this; // enables cascading
```

Returning ***this** enables cascading function calls

```
// Set the hour value
```

```
Time &Time::setHour( int h ) {
```

```
    hour = ( h >= 0 && h < 24 ) ? h : 0;
```

```
    return *this; // enables cascading
```

```
// Set the minute value
```

```
Time &Time::setMinute( int m ) {
```

```
    minute = ( m >= 0 && m < 60 ) ? m : 0;
```

```
    return *this; // enables cascading
```

```
}
```

```
// Set the second value
```

```
Time &Time::setSecond( int s ) {
```

```
    second = ( s >= 0 && s < 60 ) ? s : 0;
```

```
    return *this; // enables cascading
```

```
}
```

```
int Time::getHour() const { return hour; } // Get the hour value
```

```
int Time::getMinute() const { return minute; } // Get the minute value
```

```
int Time::getSecond() const { return second; } // Get the second value
```

```
// Display military format time: HH:MM
```

```
void Time::printMilitary() const {
```

```
    cout << ( hour < 10 ? "0" : "" ) << hour << ":"
```

```
        << ( minute < 10 ? "0" : "" ) << minute;
```

```
}
```

...Using the this Pointer

Example of cascaded member function calls:

Member functions **setHour**, **setMinute**, and **setSecond** all return ***this** (reference to an object)

For object **t**, consider:

```
t.setHour(1).setMinute(2).setSecond(3);
```

Executes **t.setHour(1)**, returns ***this** (reference to object) and the expression becomes

```
t.setMinute(2).setSecond(3);
```

Executes **t.setMinute(2)**, returns reference and becomes

```
t.setSecond(3);
```

Executes **t.setSecond(3)**, returns reference and becomes

```
t; (Has no effect)
```

...Time Driver.cpp

// Display standard format time: HH:MM:SS AM (or PM)

void Time::printStandard() **const** {

```
    cout    << ( ( hour == 0 || hour == 12 ) ? 12 : hour % 12 )
           << ":" << ( minute < 10 ? "0" : "" ) << minute
           << ":" << ( second < 10 ? "0" : "" ) << second
           << ( hour < 12 ? " AM" : " PM" );
```

}

// Cascading member function calls together with the **this** pointer

#include <iostream>

using std::cout;

using std::endl;

#include "time.h"

int main() {

Time t;

t.setHour(18).setMinute(30).setSecond(22);

```
    cout << "Military time: ";
    t.printMilitary( );
    cout << "\nStandard time: ";
    t.printStandard( );
```

Notice cascading function calls.

```
    cout << "\n\nNew standard time: ";
    t.setTime( 20, 20, 20 ).printStandard();
    cout << endl;
```

return 0;

}

Cascading function calls. **printStandard** must be called after **setTime** because **printStandard** does not return a reference to an object. **t.printStandard().setTime();** would cause an error.

Military time: 18:30
Standard time: 6:30:22 PM

New standard time: 8:20:20 PM



03-08

Member_INITIALIZER List

03 Object Oriented Programming

... Member Initializer List

You have used initializer lists with arrays

```
int arr[] = {1,2,3,4,5};
```

The **data members** of a class can also be **initialized using** an initializer list with the **constructor**

```
class Point {  
private:  
    int x;  
    int y;  
public:  
    Point(int i = 0, int j = 0):x(i), y(j) {}  
    /* The above use of Initializer list is optional as the constructor can also  
    be written as:  
    Point(int i = 0, int j = 0) {  
        x = i;  
        y = j;  
    } */  
  
    int getX() const {return x;}  
    int getY() const {return y;}  
};  
  
int main() {  
    Point t1(10, 15);  
    cout<<"x = "<<t1.getX()<<" , ";  
    cout<<"y = "<<t1.getY();  
    return 0;  
}
```

Non-const Members

... Member Initializer List

In-line Initializer List

```
class Test {
    int marks;
    int total;

public:
    //inline with initializer list
    Test() : marks(0),total(0) {} //default constructor

    Test(int m, int t) : marks(m), total(t) {}

    int getMarks() {
        return marks;
    }

    int getTotal() {
        return total;
    }

};
```

Out-of-line Initializer List

```
class Test {
    int marks;
    int total;

public:
    Test(); //prototype
    Test(int, int); //prototype
    int getMarks() { return marks; }
    int getTotal() { return total; }

};

//out of line with initializer list
Test::Test() : marks(0), total(0) {}
Test::Test(int m, int t) : marks(m), total(t) {}
```

••• When to use Member Initializer List

You **can use** initializer list to initialize member variables, but it is not necessary

But you **must use** an initializer list for

- Initializing a **constant data member**
- Initializing a **reference member**

const Class Members

As with member functions, **data members** can also be **const**
Constant members **must be initialized using member initializer list**

... Member Initializer List

non-static const

```
#include<iostream>
using namespace std;

class Test {
    const int t;
public:
    Test(int x):t(x) {} //Initializer list must be used
    int getT() { return t; }
};

int main() {
    Test t1(10);
    cout<<t1.getT();
    return 0;
}
```

references

```
#include<iostream>
using namespace std;

class Test {
    int &cRef;
public:
    Test(int &ref):cRef(ref) {} //Initializer list must be used
    int getRef() { return cRef; }
};

int main() {
    int x = 20;
    Test t1(x);
    cout<<t1.getRef()<<endl;
    x = 30;
    cout<<t1.getRef()<<endl;
    return 0;
}
```

...const Objects

Read-only objects

Object data members can only be read, NO write/update of data member allowed

Requires all member functions be const (except constructors and destructors)

const object must be initialized (using constructors) at the time of object creation

```
const Account inv("YMCA, FL", 5555, 5000.0)
```

const property of an object goes into effect after the constructor finishes executing and ends before the class's destructor executes

So, the constructor and destructor can modify the object



03-09

Friend Functions

03 Object Oriented Programming

... Friend Functions

To declare a **friend** function

Type friend before the function prototype in the class that is giving friendship

```
friend int myFunction( int );
```

should appear in the **class giving friendship**

friend functions: Can access **private** members of another class

Properties of friendship

Friendship is granted, not taken

Not symmetric (if **B** a friend of **A**, **A** not necessarily a friend of **B**)

Not transitive (if **A** a friend of **B**, **B** a friend of **C**, **A** not necessarily a friend of **C**)

...Time Driver.cpp

// Friends can access private members of a class.

```
#include <iostream>
```

```
using std::cout;
```

```
using std::endl;
```

setA a **friend** of class **Count**
(can access **private** data).

// Modified Count class

```
class Count {
```

```
    friend void setA( Count &, int); // friend function declaration
```

```
public:
```

```
    Count() { a = 0; } // constructor
```

```
    void print() const { cout << x << endl; } // output
```

```
private:
```

```
    int a; // data member
```

```
};
```

// Can modify private data of Count because

// setX is declared as a friend function of Count

```
void setA ( Count &c, int val) {
```

```
    c.x= val; // legal: setA is a friend of Count
```

```
}
```

Changing **private** variables allowed.

```
int main() {
```

```
    Count counter;
```

```
    cout<< "counter.aafter instantiation: ";
```

```
    counter.print();
```

```
    cout<< "counter.aafter call to setAfriend function:";
```

```
    setA( counter, 8 ); // set a with a friend
```

```
    counter.print();
```

```
    return 0;
```

```
}
```

setA is defined normally
and is not a member
function of **Count**.

private data was changed.

counter.a after instantiation: 0
counter.a after call to setA friend function: 8

...friend Functions and friend Classes

friends should be used only for **limited purpose**

Too many functions declared as **friends** with **private** data access,
lessens the value of encapsulation